3DGS-RTMaterial V2.1: A Stable Interactive System for Segment-Level Appearance Editing of Gaussian Scenes

Zhaoyue Yuan Yuehai Zhou
Simon Fraser University

V2.1 stable report – August 2026

Abstract

We present V2.1 of 3DGS-RTMaterial, an interactive system that combines a pre-trained 3D Gaussian Splatting (3DGS) scene, SAGA-style feature-field segmentation, temporary spherical-harmonic (SH) and opacity edits, and NVIDIA 3D Gaussian Ray Tracing (3DGRT). The system is deliberately described as an appearance editor rather than an inverse-rendering or physically based material estimator: illumination remains baked into learned SH coefficients, and the five presets are stylized transformations. V2.1 focuses on reliability. It makes empty selections no-ops, replaces view-local SAM index voting with visibility-aware cross-view association, corrects the 3DGRT SH-degree contract, eliminates empty-sample NaNs in contrastive training, freezes scene state for background export, and composes ID channels by nearest depth. A 61,380-Gaussian GPU smoke test and 26 CPU-safe regression tests provide a reproducible stability baseline. We document the exact implemented pipeline, its limits, and a path toward intrinsic decomposition and relighting.

1 Introduction

3DGS represents a radiance field with anisotropic Gaussian primitives and achieves high-quality novel-view synthesis through ordered alpha compositing [Kerbl et al., 2023]. Its compact explicit representation is attractive for interactive editing, but it does not expose albedo, roughness, metallicity, or illumination. Appearance is stored as opacity and view-dependent SH colour.

Our objective is therefore practical: select a coherent subset of Gaussians, apply reversible appearance transformations, preview through rasterization or 3DGRT, and export deterministic results. V2.1 makes three claims only:

1. segment labels can be created interactively or from multi-view SAM masks and remain attached to 3D Gaussians;
2. segment-specific SH/opacity transformations are reversible and can be rendered by both the raster and ray-tracing paths;
3. state, export, and tests are sufficiently isolated to serve as a stable base for subsequent research.

It does *not* claim recovered physical materials, relighting, refraction, or real-time full-resolution OptiX rendering.

2 Related Work

Gaussian scene representations. 3DGS introduced anisotropic Gaussian primitives optimized from calibrated images [Kerbl et al., 2023]. 3DGRT traces those primitives through a BVH and exposes depth, opacity, and normal estimates in addition to radiance [Moenne-Loccoz et al., 2024]. 3DGUT generalizes Gaussian tracing to distorted cameras and secondary-ray use cases [Wu et al., 2025].

Language and mask supervision. SAGA distills multi-view 2D segmentation masks into a scale-aware 3D Gaussian feature field for promptable selection [Cen et al., 2023]. SAM supplies the 2D automatic masks [Kirillov et al., 2023], while CLIP supplies a shared image-text embedding used here only for coarse material-name ranking [Radford et al., 2021]. The distinction is important: click segmentation uses the learned 3D feature field; SAM-driven segmentation explicitly merges 2D masks; CLIP material detection averages scores from three nearby views.

Inverse rendering and relighting. GS-IR learns normals, material parameters, and illumination for relighting [Liang et al., 2024]; GaussianShader targets reflective appearance with a shading model [Jiang et al., 2024]; SVG-IR improves visibility-aware inverse rendering [Sun et al., 2025]. These methods motivate a future physical branch, but differ from V2.1 because they optimize intrinsic scene properties. V2.1 instead edits an already trained radiance representation without claiming decomposition.

3 System Architecture

Figure 1 shows the implemented data flow. Training produces a scene Gaussian model and a 32-dimensional contrastive feature model. The GUI owns camera, segmentation labels, material assignments, undo/redo history, and project metadata. Two render backends consume the same scene: the differentiable rasterizer during interaction, and 3DGRT for settled or offline frames.

3.1 Gaussian and camera interface

For Gaussian i, the scene stores center $\boldsymbol{\mu}_i$, log-scale $\mathbf{s}_i$, quaternion $\mathbf{q}_i$, opacity logit o_i, and SH coefficients $\mathbf{f}_i$. The adapter exposes activated scale $\exp(\mathbf{s}_i)$, normalized rotation, and opacity $\sigma(o_i)$. SAGA stores features as $N \times 16 \times 3$ for degree three; V2.1 flattens this to $N \times 48$ for the tracer

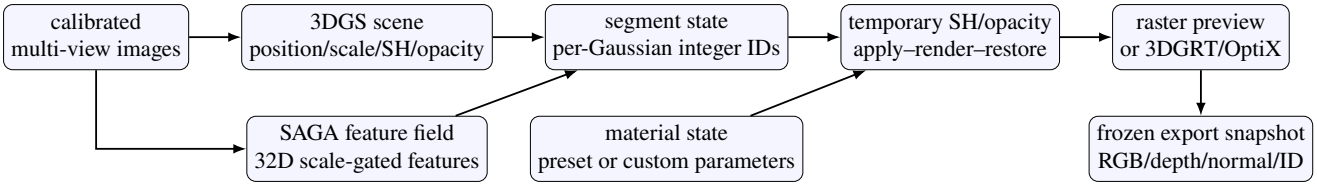

Figure 1: V2.1 architecture. Material edits are temporary transformations of learned appearance, not a recovered BRDF. Export operates on cloned tensors and copied state so subsequent GUI edits cannot change an in-flight result.

but reports active degree 3, because the 3DGRT API interprets that field as the SH *degree*, not a coefficient count. Camera rays remain in camera space and the extrinsic transform is applied exactly once inside the tracer.

3.2 Interactive render policy

Full OptiX frames are expensive on the reference pipeline. While orbiting or panning, V2.1 uses the raster backend with the same temporary material edits. On release, a dirty flag requests a full 3DGRT settle frame. This policy keeps camera control responsive without implying that both paths have identical intersection semantics.

4 Segmentation

4.1 Feature-field selection

The contrastive feature model is trained from multi-view SAM mask relations, following SAGA's scale-conditioned formulation. At run time, click prompts query the rendered feature map and produce a per-Gaussian boolean selection. Confirmed segments are encoded as integers in the scene mask; label 1 denotes unsegmented Gaussians and segment k uses encoded value $k + 1$. In V2.1 an empty boolean selection returns immediately. Earlier behavior inverted an empty mask and could silently segment the entire scene.

4.2 Explicit multi-view SAM association

SAM mask indices are local to an image. Consequently, mask 17 in two views has no guaranteed semantic correspondence. V2.1 constructs correspondence from shared visible 3D support:

1. project Gaussian centers into each calibrated image and reject points outside the frustum;
2. apply a per-pixel center z-buffer, retaining the nearest depth within a small tolerance to suppress obvious occluded votes;
3. because automatic SAM masks overlap, assign a point to the smallest containing mask in that view, yielding at most one vote per view;
4. treat every (view, local-mask) pair as a distinct node and union nodes in nearby views when their 3D intersection satisfies either IoU ≥ 0.15 or containment ≥ 0.50;
5. assign each Gaussian to the cross-view component with the largest number of distinct-view votes, requiring at least two views and ten points per output segment.

This is a geometry-based association heuristic, not learned instance tracking. The center z-buffer does not model full projected Gaussian footprints, so thin structures and heavy occlusion remain difficult.

4.3 Three-view CLIP material ranking

For coarse material suggestions, the viewer renders yaw offsets of $-10°$, $0°$, and $+10°$, forms a masked crop from the original RGB image, and averages image–text similarity for each material prompt:

$$\bar{s}_m = \frac{1}{3}\sum_{v=1}^{3}\langle \mathbf{e}_v^{\text{image}}, \mathbf{e}_m^{\text{text}}\rangle. \tag{1}$$

A minimum score and top-two margin reject uncertain predictions. This is multi-view score aggregation, not a 3D material estimator.

5 Appearance Editing

The editor snapshots original DC SH, higher-order SH, and opacity tensors. For each render it restores the snapshot, applies all active assignments, renders, and restores again in a `finally` block. Let $\mathbf{c}$ be linear RGB decoded from the degree-zero coefficient. Custom parameters apply tint $\mathbf{t}$, saturation a, higher-order gain g, opacity scale p, and strength λ. In simplified form,

$$\mathbf{c}' = (1-\lambda)\mathbf{c} + \lambda\ \text{sat}_a(\mathbf{c}\odot\mathbf{t}), \tag{2}$$

$$\mathbf{f}'_{>0} = [1-\lambda+\lambda g]\mathbf{f}_{>0}, \tag{3}$$

$$\alpha' = \text{clamp}(\alpha[1-\lambda+\lambda p]). \tag{4}$$

Table 1 lists the discrete presets. These controls change baked radiance statistics and transparency; "Metal" and "Glass" are UI names, not measured physical classes.

Table 1: Implemented V2.1 preset transformations.

Preset	DC colour	SH > 0	opacity
Default	unchanged	$1.00\times$	$1.00\times$
Metal	95% desaturated	$1.50\times$	$1.00\times$
Glass	cool tint	$1.00\times$	$0.28\times$
Plastic	saturation $1.50\times$	$0.15\times$	$1.00\times$
Matte	unchanged	$0.00\times$	$1.00\times$

6 Stable State and Export

Project files store the segmentation tensor, labels, material parameters, hidden segments, camera, iterations, and relevant UI state. Undo/redo uses bounded snapshots for semantic editing operations.

Background export must not observe mutable GUI state. V2.1 clones the six Gaussian parameter tensors used by rendering, clones the integer scene mask, copies material assignments and camera state, and derives visibility before starting the worker. A dedicated renderer and material editor operate only on that frozen snapshot. This increases temporary VRAM usage but makes output independent of edits made after export begins.

RGB(A), normals, and depth come directly from a snapshot render. Segmentation and material ID channels render each labelled subset and compose its integer ID using the nearest finite positive depth with non-zero opacity. The previous maximum-opacity rule could select a farther opaque segment over a nearer translucent one. ID export remains $O(S)$ BVH builds for S segments; a tracer that returns a primitive or semantic ID in one pass is the preferred future solution.

7 Numerical Stability and Verification

Contrastive training forms positive and negative pair sets whose cardinality can be zero. V2.1 avoids division by zero during random subsampling and defines the mean of an empty selected term as a differentiable zero. Empty diagnostic cosine sets are handled identically. The loss therefore remains finite when a mini-batch has no consistent positive or negative pairs.

CI runs test discovery over all test files, not a single historical module. The 26 CPU-safe tests cover camera rays, tiled coordinates, invalid normals, material transforms, CLIP aggregation, multi-view association primitives, training empty-set behavior, export cancellation and snapshots, depth-ordered IDs, project migration, scoped paths, undo/redo, visibility caching, and CLI startup. The full suite passes under Python 3.10 and CPU PyTorch. Runtime checks also pass for CUDA rasterizers, PyTorch3D, SAM, CLIP, SlangTorch, and 3DGRT on the evaluation machine.

8 61k-Gaussian GPU Smoke Test

The bundled smoke scene contains 61,380 Gaussians. We evaluated a 160×90 frame on an NVIDIA GeForce RTX 5090 (32,088.5 MiB), Python 3.10.19, PyTorch 2.10.0+cu128, and CUDA runtime 12.8. The 3DGRT extension was compiled before timing; JIT compilation is an installation cost and is not included. CUDA synchronization brackets every measured operation. Table 2 reports one BVH build, the first render, and ten subsequent stable renders.

All RGB, normal, depth, and opacity tensors were finite, and opacity contained visible hits. These numbers validate the included small-scene path only; they must not be extrapolated to million-Gaussian scenes or higher resolutions.

Table 2: 61,380-Gaussian OptiX smoke results at 160×90.

Metric	Result
BVH build	42.24 ms
First frame	44.04 ms
Stable mean	9.15 ms (109.23 FPS)
Stable median	9.19 ms
Stable P95	9.67 ms
Stable range	8.74–9.67 ms
Peak allocated VRAM	32.73 MiB
Peak reserved VRAM	54.00 MiB

Figure 2: Representative baseline (left) and segment-level appearance edits (right). The visual change is produced by SH/opacity transformations.

9 Limitations

- SH edits do not separate reflectance from illumination and cannot provide physically correct relighting, shadows, refraction, or energy conservation.
- Gaussian center visibility is an approximation. Explicit SAM fusion can merge similar adjacent objects or fragment objects under occlusion.
- Per-segment ID export is depth-correct but computationally linear in the number of segments.
- A frozen GPU snapshot can approach twice the scene parameter memory during export. Very large scenes may require CPU staging or chunking.
- Reference 3DGRT at the evaluated resolution is a settle/offline path; responsive interaction depends on raster preview.

10 Roadmap

The next engineering release should expose primitive IDs directly from the tracer, add a GPU integration test to CI on a controlled runner, replace the center z-buffer with rendered depth/coverage tests, and benchmark representative million-Gaussian scenes. The research branch should preserve V2.1's stylized mode while adding explicit albedo, roughness, metallicity, and environment lighting. GS-IR and SVG-IR provide useful decomposition objectives, while 3DGUT offers a route to secondary rays. A later TRON-like residual model could capture transport not explained by the explicit parameters, but only after quantitative relighting datasets and held-out-view metrics are established [Perel et al., 2026].

11 Conclusion

V2.1 turns a broad prototype into a defensible stable baseline. Its contribution is not a new physical material model;

it is a coherent integration of 3D selection, reversible Gaussian appearance editing, ray-traced inspection, deterministic export, and regression-tested failure handling. The report and benchmark deliberately separate implemented facts from future research claims.

References

Jiazhong Cen, Jiemin Fang, Chen Yang, Lingxi Xie, Xiaopeng Zhang, Wei Shen, and Qi Tian. Saga: Segment any 3d gaussians. *arXiv preprint arXiv:2312.00860*, 2023.

Yingwenqi Jiang, Jiadong Tu, Yuan Liu, Xifeng Gao, Xiaoxiao Long, Wenping Wang, and Yuexin Ma. Gaussianshader: 3d gaussian splatting with shading functions for reflective surfaces. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

Alexander Kirillov, Eric Mintun, Nikhila Ravi, et al. Segment anything. In *IEEE/CVF International Conference on Computer Vision*, 2023.

Zhihao Liang, Qi Zhang, Ying Feng, Ying Shan, and Kui Jia. Gs-ir: 3d gaussian splatting for inverse rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

Nicolas Moenne-Loccoz, Ashkan Bhatt, Orazio Lang, Ruilong Sun, Danda Pani Paudel, Luc Van Gool, and Sanja Fidler. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics*, 2024.

Or Perel, Hassan Abu Alhaija, Zian Wang, Jacob Munkberg, Matan Atzmon, Sanja Fidler, and Masha Shugrina. Tron: Tracing rays to orchestrate a neural renderer for 3d gaussian reconstructions. *arXiv preprint arXiv:2606.11314*, 2026.

Alec Radford, Jong Wook Kim, Chris Hallacy, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Hanxiao Sun, Yupeng Gao, Jin Xie, Jian Yang, and Beibei Wang. Svg-ir: Spatially varying gaussian splatting for inverse rendering. *arXiv preprint arXiv:2504.06815*, 2025.

Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moenne-Loccoz, and Zan Gojcic. 3dgut: Enabling distorted cameras and secondary rays in gaussian splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.